

ADULT INCOME CLASSIFICATION

End-to-End Machine Learning Project

Final Consolidated Report — Week 1, Day 1 through Day 5

Includes: full pipeline walkthrough, final results, model interpretation, production inference plan, and a consolidated log of issues/errors found during review.

Table of Contents

Executive Summary	3
1. Problem Definition, EDA & Baselines (Day 1)	3
2. Data Preparation (Day 1–2)	4
3. Model Development & Baseline Results (Day 2)	5
4. Model Tuning (Day 3–4)	6
5. Diagnostics: Overfitting, Underfitting & Calibration (Day 4)	6
6. Final Validation & Day 5 Deliverables	7
7. Limitations & Future Improvements	8
8. Conclusion	8
9. Consolidated Mistakes & Issues Log (Day 1–5)	9
10. Appendix A — Environment & Reproducibility Notes	10
11. Appendix B — Day-by-Day Scoring History	11

Executive Summary

This report consolidates the complete Week 1 machine learning workflow on the Adult (Census Income) dataset, from initial problem framing and EDA on Day 1 through final validation, interpretation, inference, and documentation on Day 5. It brings together the numeric results actually produced at each stage, and — separately — a full log of the mistakes, gaps, and inconsistencies identified while reviewing each day's submission. The intent is to give one honest, end-to-end picture of the project: what was done correctly, what numbers can be trusted, and what still needs to be fixed before the project can be called production-ready.

Across the four days reviewed so far, submission quality was mostly strong but uneven: Day 1 (85/100) and Day 2 (85/100) were both solid, execution-verified submissions with only minor documentation and reproducibility gaps; Day 3 dropped to 65/100 due to an unexecuted statistical test and a write-up conclusion that contradicted its own printed table; Day 4 scored very low (5.5/100) because the calibration task was never executed and the best-performing model from Day 3 (Gradient Boosting) was never tuned. Day 5's scenario — final validation, error analysis, interpretation, inference, and documentation — has not yet been executed, and this report flags that several Day 5 tasks are at risk of inheriting Day 4's unresolved gaps if those are not fixed first.

Overall project status: functional and partially verified, but not yet production-ready. Section 9 ("Consolidated Mistakes & Issues Log") lists every issue found, its severity, and what is needed to close it.

1. Problem Definition, EDA & Baselines (Day 1)

The project's objective is to predict whether an individual's annual income exceeds \$50,000 using the UCI Adult (Census Income) dataset — a binary classification task on demographic and employment attributes (age, education, occupation, hours worked per week, capital gains/losses, marital status, and related fields). This kind of income-threshold prediction is a standard benchmark for evaluating preprocessing of mixed categorical/numeric data, class-imbalance handling, and the trade-off between interpretable and high-capacity models — all of which this project exercises in sequence. It also mirrors real business use cases such as targeted marketing outreach or credit-risk screening, where the cost of contacting or approving the wrong person is asymmetric.

1.1 Business objective & success metric

The positive class is defined as $\text{income} > \$50,000/\text{year}$. The stated business objective is to maximize precision among the individuals the model flags as high-income, so that any downstream action taken on a positive prediction (e.g. targeted outreach) is not wasted on people who don't actually qualify. In non-technical terms, the goal was framed as: "we want to build a tool that flags likely high earners so we don't waste outreach effort on people who aren't — even if that means we miss a few actual high earners along the way."

The measured class base rate (share of positives) was approximately 23.9%, meaning roughly one in four individuals in the dataset earns above \$50,000. Given the stated precision-first objective, this base rate matters directly: a model that simply predicts the majority class (" $\leq 50K$ " for everyone) would already reach an accuracy equal to the negative class rate, roughly 76.1%, while being completely useless for the actual business goal, since it would never identify a single high earner.

1.2 Exploratory data analysis

The dataset was loaded via `fetch_openml('adult', version=2, as_frame=True)`, which avoids two classic raw-file gotchas: the leading-space variant of missing values (" ?") and the trailing-period inconsistency in the raw `adult.test` target labels (" $>50K.$ " vs " $>50K$ "). Missing-value markers were converted to NaN prior to any modeling. EDA covered dataset shape and column types, numeric summary statistics, categorical value counts, and both histogram and bar-plot visualizations of the key fields, alongside a saved summary table of positive/negative class counts and at least three notable feature distributions (age, hours-per-week, and capital-gain were all confirmed to be skewed, which directly motivated the median-imputation and log-transform choices used later in preprocessing and feature engineering).

1.3 Reproducible train / dev / test splits

A three-way stratified split was created: approximately 70% training, 10% dev (for fast iteration without touching the test set), and 20% hold-out test, all stratified on the target and using a fixed random_state. The positive rate was confirmed consistent across all three splits at approximately 23.9%, which is the correct reproducibility check — if the splits weren't properly stratified, the class balance could drift between train, dev, and test, and any metric comparison across days would no longer be a fair apples-to-apples comparison. The hold-out test set was explicitly set aside as untouched until final evaluation; using it during hyperparameter tuning would let information from the test set leak into model-selection decisions, producing an optimistic, unreliable estimate of real-world performance.

1.4 Simple baselines

Two baselines were implemented and evaluated on the hold-out test set:

- Majority-class predictor — always predicts the negative (" $\leq 50K$ ") class. Given the measured 23.9% positive rate, this baseline's accuracy is mathematically fixed at the negative class rate (~76.1%), but its recall and F1 for the positive class are 0 — it can never identify a single high earner, which makes it unacceptable against the stated precision-oriented business objective despite its deceptively high accuracy.
- Single-feature rule-based classifier — flags an individual as high-income if education-num ≥ 13 (i.e. at least a bachelor's degree) or capital-gain > 0 , a rule chosen because both fields showed a strong univariate association with the positive class during EDA. Accuracy, precision, recall, F1, ROC AUC, and PR AUC were computed for this rule and saved to a metrics table alongside a confusion matrix, giving a genuine (if simple) content-aware baseline to beat with later models.

The full baseline metrics table (accuracy / precision / recall / F1 / ROC AUC / PR AUC, plus confusion matrices) was correctly computed, saved to CSV, and confirmed present and well-structured by review; this report does not re-quote every individual baseline number verbatim, since the emphasis of Day 1 was methodology and reproducibility rather than final performance — the meaningful, review-verified numeric results begin with the Day 2 models in Section 3.

1.5 Initial error analysis

At least ten false positives and ten false negatives from the rule-based baseline were sampled and inspected for patterns, and an aggregated error_summary.csv (average age, hours-per-week, and education-num broken down by error type) was produced — a genuinely useful, review-confirmed pattern-analysis artifact, not just a manual eyeball check. Between four and six concrete data/feature issues to address on subsequent days were identified (e.g. handling of missing categorical values, encoding strategy for high-cardinality categorical fields, and the skew in numeric fields like capital-gain), which directly seeded the preprocessing and feature-engineering decisions made on Day 2 and Day 3. The write-up closed by naming ROC AUC / F1 as the primary metric to optimize for the remainder of the week, justified against the baseline numbers actually produced — precision was reframed as a secondary, threshold-tunable concern rather than the sole optimization target once it became clear a single fixed rule was too blunt an instrument on its own.

One scope note carried forward from Day 1: the error analysis was performed only against the rule-based baseline, not cross-checked against the majority-class baseline as well. This is reasonable given the majority baseline never predicts a positive at all (so it has no false positives to sample), but it does mean the false-negative pattern analysis reflects only one baseline's blind spots, not two.

2. Data Preparation (Day 1–2)

2.1 Preprocessing pipeline

Missing values encoded as "?" were converted to NaN. A single ColumnTransformer was built and reused unchanged for every model in every subsequent day:

- Numeric branch: SimpleImputer(strategy="median") → StandardScaler
- Categorical branch: SimpleImputer(strategy="most_frequent") → OneHotEncoder(handle_unknown="ignore")

Median imputation was chosen over mean because several numeric fields (notably `capital_gain`) are heavily right-skewed, so the median is far less distorted by outliers. `OneHotEncoder` was chosen over ordinal/label encoding because none of the categorical fields (`workclass`, `occupation`, `marital status`, etc.) have a natural order, and `handle_unknown="ignore"` protects inference-time predictions against unseen categories.

2.2 Feature engineering (Day 3)

Eight engineered features were added on Day 3 through a leak-safe `FunctionTransformer` that uses only current-row data (no target-encoding, no cross-row aggregation): age buckets, hours-per-week bins, a `capital_gain > 0` flag, $\log(\text{capital_gain} + 1)$, a higher-education boolean, an `education_num × hours_per_week` interaction term, and two additional engineered fields, each logged with a one-line justification and a mutual-information score in a feature dictionary.

3. Model Development & Baseline Results (Day 2)

Two baseline pipelines (preprocessing + estimator) were trained on the training split only and scored once on the untouched hold-out test set:

Model	Accuracy	ROC AUC
Logistic Regression	0.8527	0.9039
Decision Tree	0.8183	0.7522

Logistic Regression was the stronger baseline on both metrics. The Decision Tree's top-3 splits were inspected manually and were logically sensible (splitting first on `capital_gain` and education-related fields), but its lower ROC AUC and visible train/test gap flagged early overfitting risk that carried into later tuning decisions.

4. Model Tuning (Day 3–4)

4.1 Cross-validated model comparison (Day 3)

Three models were compared under identical preprocessing with StratifiedKFold (k=5):

Model	F1 (5-fold CV)	ROC AUC (5-fold CV)
Gradient Boosting	0.712	0.928

Gradient Boosting was the clear winner of the three-way comparison and was the intended lead candidate for hyperparameter tuning on Day 4.

4.2 Feature selection check

Selection method	Logistic Regression F1
No selection (all features)	0.6744
SelectKBest, k = 100	0.6717

All features were retained for tuning, since no SelectKBest variant tested beat the no-selection baseline.

4.3 Hyperparameter search (Day 4)

RandomizedSearchCV was run with F1 as the optimization target, StratifiedKFold(5), and n_jobs=-1:

Model	Test F1 (tuned)	Tuned?
Logistic Regression	0.674	Yes
Random Forest	0.691	Yes
Gradient Boosting	0.712 (carried over, untuned)	No

Random Forest is the strongest model that actually completed hyperparameter search. Gradient Boosting still holds the best raw score of the project (0.712) but that number is from Day 3's default configuration — it was never tuned, so its true ceiling is unknown. This is treated as an open, unresolved gap (see Section 9).

5. Diagnostics: Overfitting, Underfitting & Calibration (Day 4)

Learning curves and the effect of regularization strength (C for Logistic Regression, max_depth / min_samples_leaf for tree-based models) were specified as required diagnostics. Random Forest and Logistic Regression diagnostics were produced; the written interpretation attached to them, however, largely restated generic textbook definitions of overfitting/underfitting rather than a conclusion tied to the specific curves produced, so the diagnostic conclusion for this project should be treated as provisional rather than confirmed.

Probability calibration (Brier score, calibration plot, CalibratedClassifierCV where needed) and classification-threshold tuning were required Day 4 deliverables. Neither has a recorded numeric result: no Brier score, no chosen threshold, and no before/after confusion matrix exist for this project at the time of this report. This is the most significant unresolved gap in the entire project and blocks a fully correct Day 5 final validation (see Section 6.1 and Section 9).

6. Final Validation & Day 5 Deliverables

6.1 Task 1 — Final model validation (status: at risk)

Day 5 requires reloading the saved Day 4 pipeline, confirming it runs correctly on unseen data, and re-reporting accuracy, precision, recall, F1, ROC AUC / PR AUC, and Brier score on the untouched test set, alongside a final comparison table against the shortlisted models. Because calibration was never completed on Day 4, the Brier score component of this task cannot yet be filled in with a verified number, and because Gradient Boosting was never tuned, the "final model" selected for this stage would currently have to be an untuned model or a tuned runner-up — neither is fully satisfying. Recommendation: return to Day 4, tune Gradient Boosting, and complete calibration before treating any Day 5 final-validation number as trustworthy.

6.2 Task 2 — Model behavior & error analysis

A confusion matrix, false-positive/false-negative breakdown, and an assessment of which error type is more costly are required. Based on the class distribution of the Adult dataset (roughly 3:1 in favor of the " $\leq 50K$ " class), false negatives — high earners incorrectly classified as low earners — are the error type most likely to dominate unless the classification threshold is explicitly tuned, which ties this task directly back to the missing Day 4 threshold-selection work.

6.3 Task 3 — Feature & model interpretation

For Logistic Regression, coefficients should be mapped back to one-hot feature names via `get_feature_names_out` and ranked by magnitude; for the tree-based models, feature importances should be reported and compared against the coefficient ranking for consistency. Day 2's Decision Tree interpretation already produced a sensible top-3 split list (capital_gain and education-related fields dominating), which is a useful sanity check to carry into this task.

6.4 Task 4 — Production-ready inference

The inference workflow should load the saved joblib/pickle artifact, accept new raw input rows, let the pipeline apply preprocessing automatically, generate `predict_proba` output, apply the selected threshold, and return a final class label — without manually repeating any preprocessing step outside the pipeline. This is straightforward to build once a final pipeline is confirmed, but should not be finalized until the threshold from the (currently missing) Day 4 calibration step is available; otherwise the inference code will default to sklearn's implicit 0.5 threshold, which was never validated as the right business threshold for this task.

6.5 Task 5 — Final documentation

A README should capture: project objective, dataset and target description, feature engineering, preprocessing steps, models tested, tuning approach and best parameters, selected final model, classification threshold, final test performance, important features, known limitations, and reproduce/inference instructions, along with library versions. A README of this shape already accompanies this report; it should be updated once the Section 9 open items are closed.

6.6 Task 6 — Final report & presentation

This document fulfills the structural requirement of a Day 5 final report (problem definition, data preparation, model development, tuning, diagnostics, final results, interpretation, production readiness, and limitations), while being explicit that some numeric fields cannot yet be finalized because upstream Day 4 work is incomplete.

7. Limitations & Future Improvement

- Gradient Boosting, the best-performing model in the project, has never been hyperparameter-tuned. This should be the first fix before any "final model" claim is made.
- No calibration or threshold-selection numbers exist yet; the project cannot claim a validated decision threshold.
- The paired statistical significance test between the top two models (Day 3, Task 4) was never executed, so claims about one model being "meaningfully better" than another are not yet statistically supported.
- A small, unexplained accuracy drift (0.8527 vs 0.8524) between two runs of the same Day 2 pipeline suggests the pipeline is not fully bit-reproducible despite a fixed `random_state` — worth a dedicated reproducibility audit.
- With more data or additional features (e.g. industry-level aggregates, geographic cost-of-living indices), the ceiling on F1/ROC AUC likely has room to grow beyond the current best of 0.712 / 0.928.
- Subgroup performance (by sex, race, or age band) was never explicitly analyzed; before any production use, a fairness/subgroup error-rate check is recommended given the sensitivity of income prediction.

8. Conclusion

The project demonstrates a complete, largely correct machine learning workflow: leak-safe preprocessing, sensible feature engineering, appropriate use of cross-validation, and a real (if partial) hyperparameter search, all evaluated against a consistently held-out test set. The numeric results that do exist are internally consistent and were verified against independent recomputation. However, the project is not yet finished: the strongest model was never tuned, calibration and threshold selection were never executed, and one write-up's conclusion directly contradicted its own results table. Section 9 below gives the complete, itemized list of what needs to be fixed, in priority order, before this can be called a finished, production-ready deliverable.

9. Consolidated Mistakes & Issues Log (Day 1–5)

Every issue identified while reviewing the project's notebooks, PDFs, and write-ups is listed below, in the order it was found, with a severity rating and what is required to close it.

#	Day / Task	Issue	Severity	Fix required
1	Day 1 — Deliverable	No 1-page PDF summary found in the submission — only notebooks and CSV outputs were present.	Major	Produce and submit the missing PDF covering problem framing, chosen metric, baseline results, and error analysis.
2	Day 1 — Documentation	README is stale: it states "Tasks 1–3 completed" even though Tasks 4 and 5 are fully done in the repo.	Minor	Update the README status line to reflect all 5 tasks as complete; keep README status in sync with actual repo contents going forward.
3	Day 1 — Task 5	Error-pattern analysis was performed only against the rule-based baseline, not cross-checked against the majority-class baseline.	Cosmetic	Optionally note explicitly why the majority baseline was excluded (it produces no positive predictions, so no false positives exist to sample) rather than leaving the narrower scope unexplained.
4	Day 2 — Task 3 vs Task 5	Logistic Regression accuracy differs across two runs of the same pipeline (0.8527 vs 0.8524) despite a fixed <code>random_state</code> .	Minor	Audit pipeline for any non-deterministic step (e.g. solver, shuffling) and re-run end-to-end to confirm bit-identical output.
5	Day 2 — Task 3 notebook	<code>model_evaluation.ipynb</code> shows <code>execution_count</code> : None despite containing real, correct saved outputs.	Cosmetic	Re-save/re-export the notebook so execution counts are populated; not a genuine execution gap, but should be cleaned up.
6	Day 3 — Task 4	<code>Statistical_comparison.ipynb</code> (paired t-test / Wilcoxon between top 2 models) is entirely unexecuted — no output cells.	Major	Actually run the test, report the p-value, and interpret it against the CV score differences.
7	Day 3 — Task 5	<code>Comparison.ipynb</code> write-up claims <code>SelectKBest</code> gave the strongest F1, but its own printed table shows no-selection F1 (0.6744) beating every <code>SelectKBest</code> variant (best 0.6717).	Major	Correct the recommendation text to match the printed numbers: keep all features, do not use <code>SelectKBest</code> .
8	Day 3 — Deliverable	<code>Day3-Report.pdf</code> is corrupted / not a valid PDF file.	Major	Regenerate and validate the PDF opens correctly before resubmission.
9	Day 3 — Deliverable	<code>Week1-Day3.pdf</code> (the valid PDF) is generic and descriptive; it contains no actual computed numbers — no F1 means, no p-values, no feature importances.	Major	Rewrite the write-up to state the actual numbers produced in the notebooks, not just the methodology.
10	Day 4 — Task 4	Calibration and threshold tuning are entirely unexecuted: no Brier score, no chosen threshold, no before/after confusion matrix; the PDF only describes the steps generically.	Critical	Run <code>CalibratedClassifierCV</code> / calibration curve, report an actual Brier score, pick and justify a threshold, and show before/after confusion matrices.
11	Day 4 — Task 2	Gradient Boosting — the best-performing model from Day 3 (F1 0.712) — was never run through <code>RandomizedSearchCV</code> ; only Logistic Regression and Random Forest were tuned.	Critical	Run <code>RandomizedSearchCV</code> over Gradient Boosting's key hyperparameters (<code>learning_rate</code> , <code>max_iter</code> ,

				max_depth, l2_regularization) and re-evaluate on the test set.
12	Day 4 — Task 3	The overfitting/underfitting diagnosis is written as generic textbook statements rather than a conclusion tied to the project's own learning curves.	Moderate	Re-write the diagnosis to reference the actual curve shapes and specific train/validation gaps observed for this project's models.
13	Day 5 — Task 1 (at risk)	Final validation depends on a tuned Gradient Boosting model and a validated calibration/threshold, neither of which currently exists.	Critical (blocking)	Resolve issues #10 and #11 first; only then treat Day 5's final metrics table as trustworthy.

9.1 Severity summary

Severity	Count	Meaning
Critical	3	Blocks a correct final result; must be fixed before Day 5 sign-off.
Major	5	Materially wrong or missing analysis/deliverable; undermines a specific claim or requirement.
Moderate	1	Weak or generic reasoning; doesn't invalidate results but reduces report quality.
Minor	2	Small inconsistency or documentation lag, unlikely to affect conclusions.
Cosmetic	2	Presentation/scope/export issue only; no effect on correctness.

Priority order for closing these items: fix #10 and #11 first (both Critical, and both block a trustworthy Day 5 final validation), then #1, #6, #7, #8, #9 (Major — a missing deliverable plus mostly missing or contradictory analysis/documentation), then #12 (Moderate), and finally #2, #3, #4, #5 (Minor/Cosmetic, safe to defer).

10. Appendix A — Environment & Reproducibility Notes

The following practices were applied throughout the project to keep results reproducible, and should be preserved in any future work on this pipeline:

- A single train/test split was created on Day 1 and never touched again except for final scoring — every metric in this report comes from that same hold-out set.
- `random_state` was fixed on every estimator, every `train_test_split` call, and every cross-validation splitter (StratifiedKFold) throughout Days 2–4.
- All preprocessing lives inside a single sklearn Pipeline / ColumnTransformer object per model, so inference never requires manually repeating a transformation step outside the saved pipeline.
- Feature engineering functions were written to depend only on the current row, with no aggregation across rows and no use of the target column, to avoid leakage into cross-validation folds.
- Final pipelines were serialized with joblib so the exact fitted preprocessing + estimator object can be reloaded without re-running training.

Recommended environment for reproducing this project (to be finalized once Section 9's open items are closed and pinned in requirements.txt):

Package	Role in project
python	Base interpreter
pandas / numpy	Data loading, cleaning, and array operations
scikit-learn	Preprocessing pipeline, models, cross-validation, tuning
matplotlib	ROC/PR curves, calibration plots, learning curves
joblib	Saving/loading the final fitted pipeline
scipy	Paired statistical test (t-test / Wilcoxon)

Exact pinned version numbers were documented alongside the Day 4 saved pipeline per that day's Task 1 requirement, and should be copied verbatim into the project's requirements.txt rather than re-typed from memory, to avoid a silent version mismatch between the environment that produced these numbers and the environment used to reproduce them.

11. Appendix B — Day-by-Day Scoring History

For transparency, the review scores received at each stage are recorded below alongside the single biggest reason for that score. This is included so that progress (or regression) in submission quality is visible at a glance across the project, independent of the model-performance numbers reported elsewhere in this document.

Day	Score	Single biggest factor
Day 1 — Problem Definition, EDA & Baselines	85 / 100	Strong, fully executed, execution-verified submission; only gaps were a missing PDF deliverable and a stale README status line.
Day 2 — Preprocessing & Baseline Models	85 / 100	Verified, trustworthy metrics; only minor reproducibility drift and a cosmetic notebook-export issue.
Day 3 — Feature Engineering & CV Comparison	65 / 100	Statistical test never executed, and the written recommendation contradicted the notebook's own results table.
Day 4 — Tuning, Regularization & Reproducible Pipelines	5.5 / 100	Calibration/threshold task entirely unexecuted, and the best model (Gradient Boosting) was never tuned.
Day 5 — Final Validation, Interpretation & Documentation	Not yet scored	Pending completion; at risk of inheriting Day 4's unresolved Critical issues (see Section 9, item 10).

Read together with Section 9, the trend across Day 3 and Day 4 is the same failure mode repeating: a task is described in prose but never actually executed with real numeric output (the statistical test on Day 3, the calibration step on Day 4). The single most valuable habit to fix going into Day 5 is simple — every task in the scenario should end with a printed, saved number, not a paragraph describing what the number would show.